

# Conception et Développement d'une Application Mobile Multiplateforme & API Backend REST de Recrutement

**Projet : RECRUTIA**

Solution Intelligente de Mise en Relation Candidats – Recruteurs avec Validation  
Administrative des Dossiers et Notifications en Temps Réel

|                      |                                                       |
|----------------------|-------------------------------------------------------|
| Organisme / Projet : | Recrutia Mobile Platform                              |
| Réalisé par :        | Ahmed ECHIH                                           |
| Domaine :            | Génie Informatique & Développement Mobile / Fullstack |
| Technologies Clés :  | React Native (Expo), PHP Native MVC, MySQL, REST API  |
| Période de Stage :   | Année Universitaire 2025 – 2026                       |

# Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble de l'équipe encadrante et pédagogique pour leur soutien inestimable, leurs précieux conseils et leur orientation tout au long de la réalisation du projet **Recrutia**. Ce stage m'a permis d'approfondir considérablement mes compétences en ingénierie logicielle, développement mobile React Native, conception d'architectures RESTful sécurisées et gestion du cycle de vie logiciel.

# Résumé / Abstract

**Résumé (Français) :**

Le projet **Recrutia** est une application mobile moderne et performante développée avec React Native (Expo) couplée à un backend PHP REST (architecture MVC) et une base de données MySQL. L'application répond aux défis actuels du recrutement numérique en offrant un parcours d'inscription structuré en plusieurs étapes avec téléversement sécurisé de pièces justificatives (CV, NNI, Pièce d'identité), un système de validation administrative des dossiers candidats avec suivi du statut en temps réel (Polling dynamique 20s et alertes), un module de recherche et filtrage d'offres d'emploi, un système interactif de postulation avec notifications modales, et une interface ergonomique supportant le mode sombre/clair.

**Abstract (English) :**

The **Recrutia** project is a modern mobile platform developed using React Native (Expo) paired with a PHP RESTful API (MVC architecture) and a MySQL database. It addresses digital recruitment challenges by featuring a multi-step registration flow with secure document upload (Resume, National ID, Certificates), an administrative candidate file validation system with real-time status updates (20-second dynamic polling and instant alerts), an advanced job search and category filter module, an interactive application tracking system with modal notifications, and a polished dark/light mode theme support.

# Table des Matières

| Section / Chapitre | Description                                           |
|--------------------|-------------------------------------------------------|
| Chapitre 1         | Contexte Général, Problématique & Objectifs du Projet |
| Chapitre 2         | Analyse des Besoins Fonctionnels et Non-Fonctionnels  |
| Chapitre 3         | Architecture Technique, Schéma BD & Sécurité          |
| Chapitre 4         | Réalisation, Modules Frontend/Backend & Composants UI |
| Chapitre 5         | Tests, Validation, Optimisations UX & Performances    |
| Conclusion         | Bilan du Stage & Perspectives d'Évolution             |

# Chapitre 1 : Contexte Général et Présentation du Projet

## 1.1 Contexte Général

Dans un marché du travail en constante mutation, la transformation digitale des processus de recrutement est devenue un impératif stratégique pour les organisations et les candidats. Les méthodes traditionnelles basées sur des formulaires papier ou des e-mails dispersés souffrent de lenteurs administratives, d'un manque de transparence dans le suivi des candidatures et d'un risque élevé de falsification des documents d'identité et parcours.

## 1.2 Problématique

Les candidats rencontrent fréquemment des obstacles majeurs lors de leur recherche d'emploi :

- **Incertitude sur la validité du dossier** : Absences d'informations claires sur la réception et la conformité de leurs pièces justificatives (CV, pièce d'identité, certificat NNI).
- **Absence de retour en temps réel** : Délais de traitement longs sans notifications automatiques lors des changements d'état de candidature ou de dossier.
- **Expérience utilisateur dégradée** : Interfaces mobiles souvent lourdes, peu réactives et non adaptées aux préférences visuelles modernes (mode sombre).

## 1.3 Solution Proposée : L'Écosystème Recrutia

Pour répondre à ces défis, le projet **Recrutia** a été conçu sous la forme d'une plateforme d'ingénierie logicielle mobile-first complète. Recrutia permet de sécuriser le recrutement dès l'inscription via une validation rigoureuse du dossier candidat par l'administration, tout en offrant aux candidats retenus une expérience fluide de consultation, de recherche filtrée, de postulation en un clic et de suivi interactif.

# Chapitre 2 : Analyse des Besoins et Spécifications

## 2.1 Besoins Fonctionnels

| Module Fonctionnel                          | Spécifications & Fonctionnalités Réalisées                                                                                                                                            |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentification & Inscription Multi-Étapes | Formulaire guidé en 4 étapes (Identité, Informations, Parcours, Confirmation) avec téléversement obligatoire de la Photo, du CV, de la Pièce d'Identité et du Certificat NNI.         |
| Validation du Dossier Candidat              | Gestion des états de dossier (en_attente, accepte, refuse) avec saisie du motif de refus. Polling automatique toutes les 20s et popups d'alerte lors de la validation administrative. |
| Consultation & Recherche d'Offres           | Affichage dynamique des offres d'emploi, recherche multi-critères par mot-clé (titre, entreprise, lieu) et filtrage instantané par catégorie d'activité.                              |
| Postulation & Suivi Candidatures            | Envoi de candidature en un clic, annulation possible, vérification préalable des doublons et historique complet de statut (retenue / non retenue).                                    |
| Notifications Interactive                   | Tiroir modale de notifications avec gestion des statuts de lecture localisés (AsyncStorage), badges de non-lus dynamiques sur l'en-tête et bouton 'Tout lire'.                        |
| Gestion du Profil & Préférences             | Édition intégrale du profil avec remplacement atomique des documents sur le serveur, changement sécurisé de mot de passe et basculement du Thème Sombre/Clair.                        |

## 2.2 Besoins Non-Fonctionnels

- **Sécurité et Confidentialité** : Cryptage des mots de passe avec l'algorithme fort `password_hash()` (BCrypt/Argon2), isolation des fichiers dans des répertoires dédiés et nettoyage atomique des fichiers orphelins.
- **Ergonomie et Fluidité UX** : Composants modernes avec animations d'entrée et de chargement (`SplashScreen`, `HeaderMenu` à fond transparent, tiroir `SideMenuModal`).
- **Modularité et Scalabilité** : Architecture backend MVC en PHP natif sans dépendance externe lourde, garantissant des temps de réponse d'API très faibles (< 50ms).

## Chapitre 3 : Architecture Technique, Schéma BD & Sécurité

### 3.1 Architecture Système RESTful Client-Serveur

Recrutia repose sur une architecture découplée **Client-Serveur RESTful**. Le client mobile développé en **React Native / Expo** communique avec le serveur via des requêtes HTTP JSON vers le point d'entrée centralisé `backend/routes/api.php`. Le backend suit le design pattern **MVC (Modèle-Vue-Contrôleur)**.

#### Structure de la Chaîne Traitement Backend PHP :

- **api.php** : Routage dynamique basé sur le paramètre `?action=...`
- **Contrôleurs (UserController, OffreController, CandidatureController)** : Gestion de la logique métier, validation du payload et réponses HTTP JSON.
- **Modèles (User, Offre, Candidature)** : Abstraction de la base de données via **PDO (PHP Data Objects)** et requêtes préparées anti-injection SQL.
- **Stockage Fichiers (public/uploads/)** : Organisation structurée (``photos/``, ``cv/``, ``piece_identite/``, ``certificat_nni/``).

### 3.2 Modèle et Schéma de la Base de Données (SQL)

La base de données MySQL s'articule autour de 4 entités principales reliées par des clés étrangères :

| Table SQL    | Champs Principaux                                                                                                                                                              | Rôle & Clés Étrangères                                                                                        |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| utilisateurs | id, nom, prenom, email, mot_de_passe, telephone, ville, diplome, specialite, exp_annees, photo, cv, piece_identite, certificat_nni, <b>statut_dossier</b> , <b>motif_refus</b> | Stocke les données candidats et le statut d'homologation administrative ('en_attente', 'accepte', 'refuse').  |
| categories   | id, nom, icone, description                                                                                                                                                    | Classification des secteurs d'activité (Informatique, Finance, Santé, etc.).                                  |
| offres       | id, titre, description, entreprise_nom, entreprise_logo, lieu, type_contrat, salaire, categorie_id, date_publication                                                           | Offres d'emploi publiées.<br><b>FK:</b> categorie_id -> categories.id                                         |
| candidatures | id, candidat_id, offre_id, date_candidature, statut                                                                                                                            | Postulations candidats.<br><b>FK:</b> candidat_id, offre_id.<br>Statuts: 'en_attente', 'acceptee', 'refusee'. |

### 3.3 Mécanisme de Gestion Sécurisée et Atomique des Fichiers

Lors de la modification de profil (``updateProfile``), l'application applique une stratégie de nettoyage atomique développée dans la méthode privée `replaceFile()` du `UserController`. Lorsqu'un candidat soumet un nouveau document, le système enregistre le nouveau fichier sous un identifiant unique `uniqid() . '_' . time()`, puis supprime immédiatement l'ancien fichier physique du disque (``@unlink($oldPath)``). Cette approche élimine tout risque d'accumulation de fichiers orphelins sur le serveur.

## Chapitre 4 : Réalisation et Implémentation Technologique

## 4.1 Organisation Globale du Projets et Codebase

Le projet se divise clairement entre le dossier ``backend`` (API PHP) et le dossier ``frontend`` (Application Expo React Native) :

| Dossier Frontend<br>(`frontend/src/`) | Responsabilité et Composants Clés                                                                                                                                                      |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| /Screens                              | Écrans principaux: `HomeScreen`, `LoginScreen`, `RegisterScreen`, `OffreDetailScreen`, `ProfileScreen`, `EditProfileScreen`, `FavoriteScreen`, `ChangePasswordScreen`, `SplashScreen`. |
| /Register                             | Étape d'inscription modularisées: `RegisterStepIdentification`, `RegisterStepInfos`, `RegisterStepParcours`, `RegisterStepConfirm`.                                                    |
| /Composants                           | Composants réutilisables: `HeaderMenu` (fond transparent), `SearchBar`, `CategoryCard`, `JobCard`, `BottomNavigation`, `SideMenuModal`.                                                |
| /Context                              | Contextes React globaux: `AuthContext` (gestion session & statut dossier), `ThemeContext` (mode sombre/clair & traducteur i18n).                                                       |

## 4.2 Focus sur la Vérification du Statut du Dossier et le Polling (20s)

L'un des éléments majeurs de la réalisation réside dans l'intégration de la vérification dynamique du statut du dossier candidat (``statut_dossier``). Dans ``HomeScreen.jsx`` et ``AuthContext.js`` :

- **Surveillance continue** : Un intervalle temporel de 20 secondes (``setInterval(..., 20000)``) interroge l'action API `?action=user_status&id;=X`.
- **Notifications Visuelles Immédiates** : Si le statut évolue de `en_attente` vers `accepte`, une alerte native s'affiche et débloque instantanément l'accès complet aux postulations.
- **Transparence en cas de Refus** : Si le dossier est refusé, le bloc d'information affiche en rouge le motif précis saisi par l'administrateur avec un accès direct vers l'édition du profil.

## Chapitre 5 : Tests, Validation et Améliorations UX

### 5.1 Protocole de Test et Validation

Chaque module a fait l'objet d'un protocole de validation rigoureux :

| Type de Test                     | Scénario de Test                                                                                             | Résultat Obtenu                                                                           |
|----------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Tests API REST                   | Exécution des requêtes Inscription, Login, Update Profile avec Postman et requêtes HTTP multipart/form-data. | <b>SUCCÈS</b> (Réponses JSON valides, codes HTTP 200, gestion des erreurs 400/409).       |
| Tests d'Upload Fichiers          | Remplacement successif du CV et de la photo dans l'écran EditProfileScreen.                                  | <b>SUCCÈS</b> (Anciens fichiers supprimés du disque, répertoires créés dynamiquement).    |
| Tests de Polling & Notifications | Modification du statut dossier en base de données pendant que le candidat navigue sur l'application.         | <b>SUCCÈS</b> (Détection sous 20s, affichagePopup et mise à jour de la modale de notifs). |
| Tests d'Ergonomie Visuels        | Validation du Header transparent (`HeaderMenu.jsx`) et suppression des éléments perturbateurs du Splash.     | <b>SUCCÈS</b> (Superposition parfaite sans ombre parasite, rendu épuré).                  |

### 5.2 Synthèse des Améliorations UX / UI apportées

- En-tête Transparent (HeaderMenu) :** Intégration d'un prop transparent = true par défaut avec annulation des ombres (`elevation: 0`, `shadowOpacity: 0`) pour un design moderne et immersif.
- Splash Screen Épuré :** Suppression du cercle orange d'arrière-plan (`ambientGlow`) afin de focaliser l'attention de l'utilisateur sur le logo et l'animation de démarrage.
- Gestion Dynamique des Badges :** Compteur temps réel des notifications non lues avec persistance locale dans `AsyncStorage`.

## Conclusion Générale et Perspectives

La réalisation du projet **Recrutia** dans le cadre de ce stage de fin d'études a constitué une opportunité exceptionnelle d'appliquer les principes fondamentaux du génie logiciel et du développement mobile d'entreprise. L'application délivrée offre une solution robuste, sécurisée et parfaitement adaptée aux besoins des candidats et des recruteurs.

#### Bilan des Compétences Acquisées :

- Maîtrise avancée de **React Native (Expo)** et de l'architecture par Contextes (Auth, Theme, i18n).
- Conception d'une **API RESTful robuste en PHP Native MVC** avec PDO et requêtes préparées.
- Gestion efficace de l'état asynchrone, du stockage local (`AsyncStorage`) et du téléversement sécurisé de fichiers.
- Sens aigu du design UI/UX, de l'accessibilité et de l'optimisation des composants réutilisables.

#### Perspectives d'Évolution Future :

1. **Intégration de Push Notifications WebSockets / Firebase Cloud Messaging (FCM)** : Remplacer le polling 20s par une communication bidirectionnelle en temps réel.
2. **Algorithme de Matching IA des CVs** : Automatiser le rapprochement entre le profil candidat et les offres pertinentes grâce au Traitement Automatique du Langage (NLP).
3. **Géolocalisation des Offres** : Intégrer une carte interactive pour visualiser les offres d'emploi à proximité du candidat.

---

Rapport validé et généré pour le projet Recrutia • Année 2025 - 2026